

chunyuac_HW1

October 21, 2020

Chun-Yuan (Scott) Chiu

chunyuac@andrew.cmu.edu

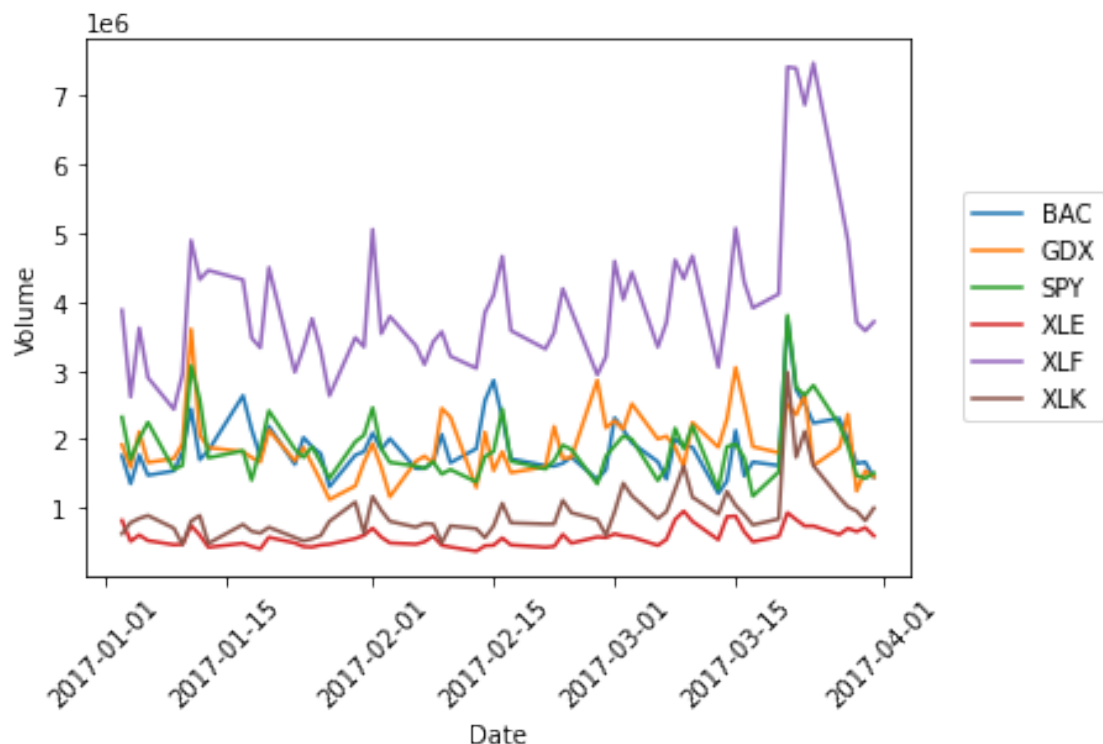
0.1 Part 1

The data with index 278303 is BAC (Bank of America Corp) on 2017/3/21. Huge trading volume happened on that date. The below plot shows the time series of the total volume of BAC and SPY (an ETF tracking S&P 500) and selected sector ETFs. In general there was a spike in volume on that date but the financials sector ETF XLF has by far the largest volume. Also, it can't be seen in this data but BAC was down almost 6% on that date while S&P 500 was only down 1.3%. Most likely there were bad news in finance or even on BAC itself.

```
[67]: import matplotlib.pyplot as plt
import pandas as pd

fulldata = pd.read_csv('q1_2017_all.csv')
fulldata['Date'] = pd.to_datetime(fulldata['Date'], format='%Y%m%d')
fulldata['Volume'] = sum([fulldata[col] for col in fulldata.columns if '000' in
    ↳ col])

data = fulldata[fulldata['Ticker'].isin(['BAC', 'XLF', 'SPY', 'XLK', 'XLE',
    ↳ 'GDX'])][['Date', 'Ticker', 'Volume', 'VolatilityRank']]
ax = sns.lineplot(x='Date', y='Volume', hue='Ticker', data=data)
ax.legend(loc='right', bbox_to_anchor=(1.25, 0.5), ncol=1)
plt.xticks(rotation=45);
```



0.2 Part 2

- As indicated below, there are 356 records on 13 variables.

```
[72]: import numpy as np
      from pandas import DataFrame

      fulldata = pd.read_csv('crsp_all.csv')
      fulldata.shape
```

[72]: (356, 13)

- The below table shows that there are 4 different TICKERs but only 2 distinct CUSIP values, as the TICKER for CUSIP M7037810 has changed twice, once in August 2003 because the company changed its name and the other in December 2014 because of a [merger](#).

```
[81]: idx = fulldata[['TICKER', 'CUSIP', 'COMNAM']].drop_duplicates().index
      fulldata.iloc[idx, :]
```

```
[81]:
```

	PERMNO	date	TICKER	COMNAM	CUSIP	BIDLO	\
0	84415	20000929	CRYS	CRYSTAL SYSTEMS SOLUTIONS	M7037810	7.875	
35	84415	20030829	BPHX	BLUEPHOENIX SOLUTIONS LTD	M7037810	3.310	

171	84415	20141231	MDSY	BLUEPHOENIX SOLUTIONS LTD	M7037810	3.200
172	84415	20150130	MDSY	MODSYS INTERNATIONAL LTD	M7037810	3.050
178	85914	20000929	BBY	BEST BUY COMPANY INC	08651610	61.000

	ASKHI	PRC	VOL	BID	ASK	SHROUT	CFACPR
0	9.125	8.3125	3792	8.0625	8.4375	7000	0.25
35	3.690	3.5400	296	3.3700	3.5400	13449	0.25
171	3.750	3.4600	834	3.4600	3.5500	17844	1.00
172	3.460	-3.0750	223	2.9500	3.2000	17844	1.00
178	70.375	63.6250	322937	63.5000	63.6875	206755	2.25

- There are 4 different values of CFACPR and no missing data in this variable, as shown in the below table.

```
[4]: fulldata['CFACPR'].drop_duplicates()
```

```
[4]: 0      0.25
     135    1.00
     178    2.25
     198    1.50
     Name: CFACPR, dtype: float64
```

- One example where CFACPR is not 1 is as below:

```
[5]: fulldata.iloc[178, :]
```

```
[5]: PERMNO      85914
     date      20000929
     TICKER      BBY
     COMNAM    BEST BUY COMPANY INC
     CUSIP      08651610
     BIDLO       61
     ASKHI      70.375
     PRC       63.625
     VOL      322937
     BID       63.5
     ASK      63.6875
     SHROUT    206755
     CFACPR     2.25
     Name: 178, dtype: object
```

- Dividing PRC by CFACPR to form the adjusted price PRCadj, there is no missing data in PRCadj.

```
[82]: fulldata['PRCadj'] = fulldata['PRC']/fulldata['CFACPR']
      np.where(fulldata['PRCadj'].isna())
```

```
[82]: (array([], dtype=int64),)
```

- There are 178 unique dates.

```
[7]: np.unique(fulldata['date']).shape
```

```
[7]: (178,)
```

```
[84]: fulldata[(fulldata['PRCadj'] < 0) & (fulldata['date']//10000 == 2014)]
```

```
[84]:
```

	PERMNO	date	TICKER	COMNAM	CUSIP	BIDLO	\
164	84415	20140530	BPHX	BLUEPHOENIX SOLUTIONS LTD	M7037810	4.1000	
168	84415	20140930	BPHX	BLUEPHOENIX SOLUTIONS LTD	M7037810	3.5501	

	ASKHI	PRC	VOL	BID	ASK	SHROUT	CFACPR	PRCadj
164	4.30	-4.100	260	4.05	4.15	11468	1.0	-4.100
168	3.81	-3.595	130	3.50	3.69	11621	1.0	-3.595

- Daily market data found in WRDS shows that on these two dates BPHX has zero volume and hence no closing price, in which case CRSP data uses a negative average of the bid and the ask prices as the closing price, as explained [here](#).

```
[87]: daily = pd.read_csv('crsp_daily.csv')
daily[(daily['CUSIP']=='M7037810') & (daily['date'].isin([20140930, 20140530]))]
```

```
[87]:
```

	PERMNO	date	TICKER	COMNAM	CUSIP	PRC	VOL
3454	84415	20140530	BPHX	BLUEPHOENIX SOLUTIONS LTD	M7037810	-4.100	0
3539	84415	20140930	BPHX	BLUEPHOENIX SOLUTIONS LTD	M7037810	-3.595	0

- There was one record where the closing bid is larger than the closing ask, known as a crossed market. According to [this paper](#), in early 2000s crossed markets happen from time to time but only last for a few seconds before they are arbitrated away. A crossed market can happen when two market makers in different trading venues post bid and ask prices at around the same time, where one's bid is larger than the other's ask. In our data the crossed market is a closing bid/ask prices. Most likely it happened a few seconds before closing and before anyone saw the arbitrage opportunity the market was closed.

```
[89]: fulldata['PRCadj'] = np.abs(fulldata['PRCadj'])
fulldata['PRC'] = np.abs(fulldata['PRC'])
fulldata[fulldata['BID'] > fulldata['ASK']]
```

```
[89]:
```

	PERMNO	date	TICKER	COMNAM	CUSIP	BIDLO	ASKHI	\
252	85914	20061130	BBY	BEST BUY COMPANY INC	08651610	51.39	55.84	

	PRC	VOL	BID	ASK	SHROUT	CFACPR	PRCadj
252	54.97	1045295	54.92	54.91	480250	1.0	54.97

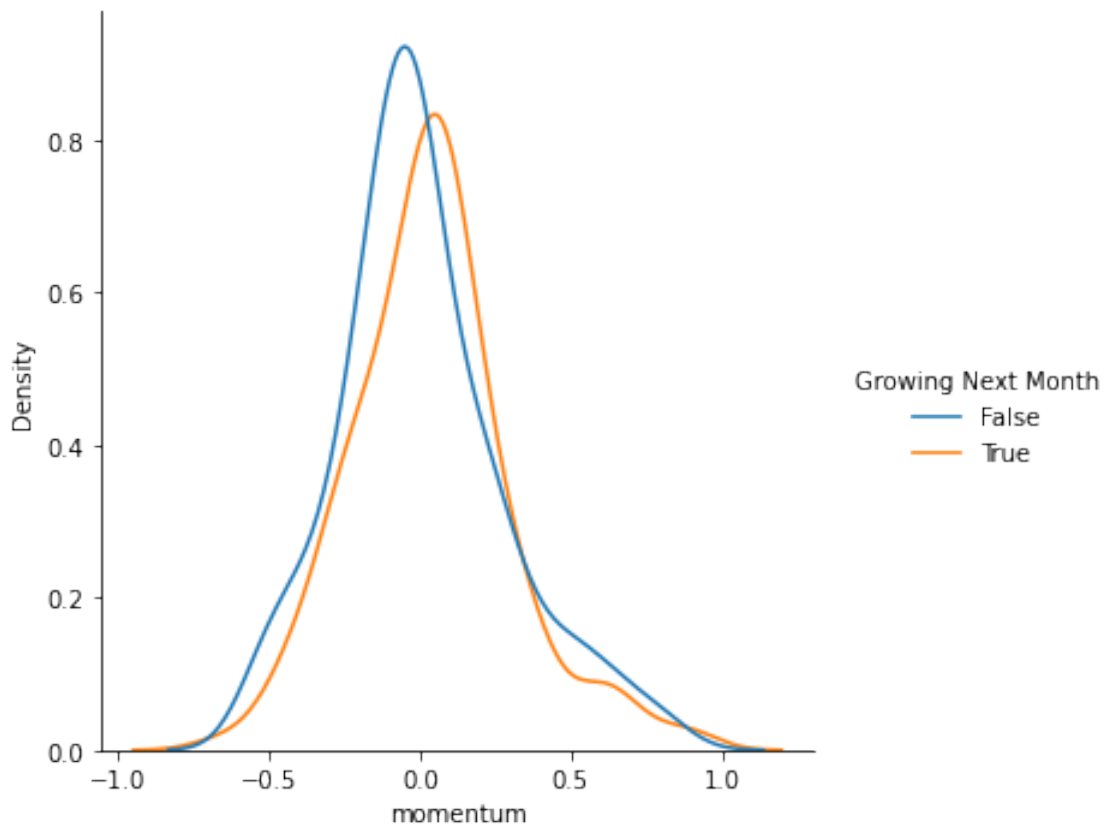
- From the below plot we see that, when stock prices are going to drop next month, the distribution of momentum is slightly skewed to the right. This is more obvious when compared to the momentum distribution when stock prices are going to go up next month.

```
[90]: %matplotlib inline
import seaborn as sns

dfs = []
for cusip, data in fulldata.groupby(['CUSIP']):
    data['momentum'] = data['PRCadj'].rolling(5).apply(lambda df: df.iloc[-2]/
    ↪df.iloc[-5] - 1)
    data['Growing Next Month'] = (data['PRCadj'].shift(-1)) > data['PRCadj']
    dfs.append(data)

fulldata = pd.concat(dfs, ignore_index=True)
sns.displot(fulldata, x='momentum', hue='Growing Next Month', kind='kde')
```

[90]: <seaborn.axisgrid.FacetGrid at 0x7fd6869fb3d0>



chunyuac_HW2

October 21, 2020

Chun-Yuan (Scott) Chiu

chunyuac@andrew.cmu.edu

46-921, Fall 2020

Due Thursday, September 17 at 9:30 AM

Chun-Yuan (Scott) Chiu, chunyuac@andrew.cmu.edu

You should submit the Jupyter notebook with your answers in the appropriate spaces. Rename the file as YOURANDREWID_HW2.ipynb and submit it via Canvas. Also submit a .pdf version of your homework.

When you are asked to create a plot, you have the flexibility to use your judgement to choose the plot you feel is most appropriate. I expect that you will take steps to make the plot clean and readable.

Question One: Go to the following website:

https://www.quandl.com/data/AAIL/AAIL_SENTIMENT-AAIL-Investor-Sentiment-Data

and download the data set. (Note that there are multiple ways of doing this, including using the “Download” button in the upper right of the screen.) Read these data into Python and create an appropriate data frame. Be sure that each column is in an appropriate form (date, factor, character, etc.) When reading in the file, do not make any changes to the data file itself. Resolve any issues using appropriate Python commands.

Answer:

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

fulldata = pd.read_csv('AAIL-AAIL_SENTIMENT.csv')
fulldata['Date'] = pd.to_datetime(fulldata['Date'], format='%Y-%m-%d').dt.date
fulldata.head()
```

```
[1]:
```

	Date	Bullish	Neutral	Bearish	Total	Bullish 8-Week Mov Avg	\
0	2020-09-10	0.237113	0.278351	0.484536	1.000000	0.270734	
1	2020-09-03	0.308017	0.274262	0.417722	1.000001	0.279641	
2	2020-08-27	0.320755	0.283019	0.396226	1.000000	0.275083	
3	2020-08-20	0.303887	0.272085	0.424028	1.000000	0.262678	

```
4 2020-08-13 0.300366 0.278388 0.421245 0.999999 0.254865
```

	Bull-Bear Spread	Bullish Average	Bullish Average + St. Dev \
0	-0.247423	0.379315	0.48007
1	-0.109705	0.379315	0.48007
2	-0.075471	0.379315	0.48007
3	-0.120141	0.379315	0.48007
4	-0.120879	0.379315	0.48007

	Bullish Average - St. Dev	S&P 500 Weekly High	S&P 500 Weekly Low \
0	0.27856	3588.11	3329.27
1	0.27856	3588.11	3444.15
2	0.27856	3481.07	3354.69
3	0.27856	3399.54	3355.46
4	0.27856	3387.89	3317.37

	S&P 500 Weekly Close
0	3398.96
1	3580.84
2	3478.73
3	3374.85
4	3380.35

Question Two: Are there any missing values in the data set? If so, describe any features/patterns in the missingness.

Answer:

First we examine where all the missing data is:

```
[2]: np.where(fulldata.isna())

[2]: (array([1279, 1280, 1281, 1282, 1283, 1284, 1285, 1286, 1286, 1286, 1286,
        1286, 1286, 1722, 1723, 1724, 1725, 1726, 1727, 1728, 1729, 1729,
        1729, 1729, 1729, 1729, 1730, 1730, 1730, 1730, 1730, 1730, 1730,
        1730, 1730]),
      array([ 5,  5,  5,  5,  5,  5,  5,  5,  1,  2,  3,  4,  5,  6,  5,  5,  5,  5,
        5,  5,  5,  1,  2,  3,  4,  5,  6,  1,  2,  3,  4,  5,  6, 10, 11,
        12]))
```

We then list the data which has at least one missing value, and present it in a heatmap to see if there are interesting patterns. The syntax here is different from what we did in the class, which I thought is a little bit more readable.

```
[3]: nadata = fulldata[fulldata.isna().apply(lambda row: row.any(), axis='columns')]
      nadata
```

[3]:

	Date	Bullish	Neutral	Bearish	Total	Bullish 8-Week Mov Avg	\
1279	1996-02-29	0.45	0.28	0.27	1.0	NaN	
1280	1996-02-22	0.49	0.22	0.29	1.0	NaN	
1281	1996-02-15	0.54	0.21	0.25	1.0	NaN	
1282	1996-02-08	0.53	0.31	0.16	1.0	NaN	
1283	1996-02-01	0.54	0.31	0.15	1.0	NaN	
1284	1996-01-25	0.36	0.40	0.24	1.0	NaN	
1285	1996-01-18	0.42	0.29	0.29	1.0	NaN	
1286	1996-01-11	NaN	NaN	NaN	NaN	NaN	
1722	1987-09-04	0.42	0.17	0.41	1.0	NaN	
1723	1987-08-28	0.52	0.18	0.30	1.0	NaN	
1724	1987-08-21	0.66	0.28	0.06	1.0	NaN	
1725	1987-08-14	0.45	0.35	0.20	1.0	NaN	
1726	1987-08-07	0.56	0.15	0.29	1.0	NaN	
1727	1987-07-31	0.26	0.48	0.26	1.0	NaN	
1728	1987-07-24	0.36	0.50	0.14	1.0	NaN	
1729	1987-07-17	NaN	NaN	NaN	NaN	NaN	
1730	1987-06-26	NaN	NaN	NaN	NaN	NaN	

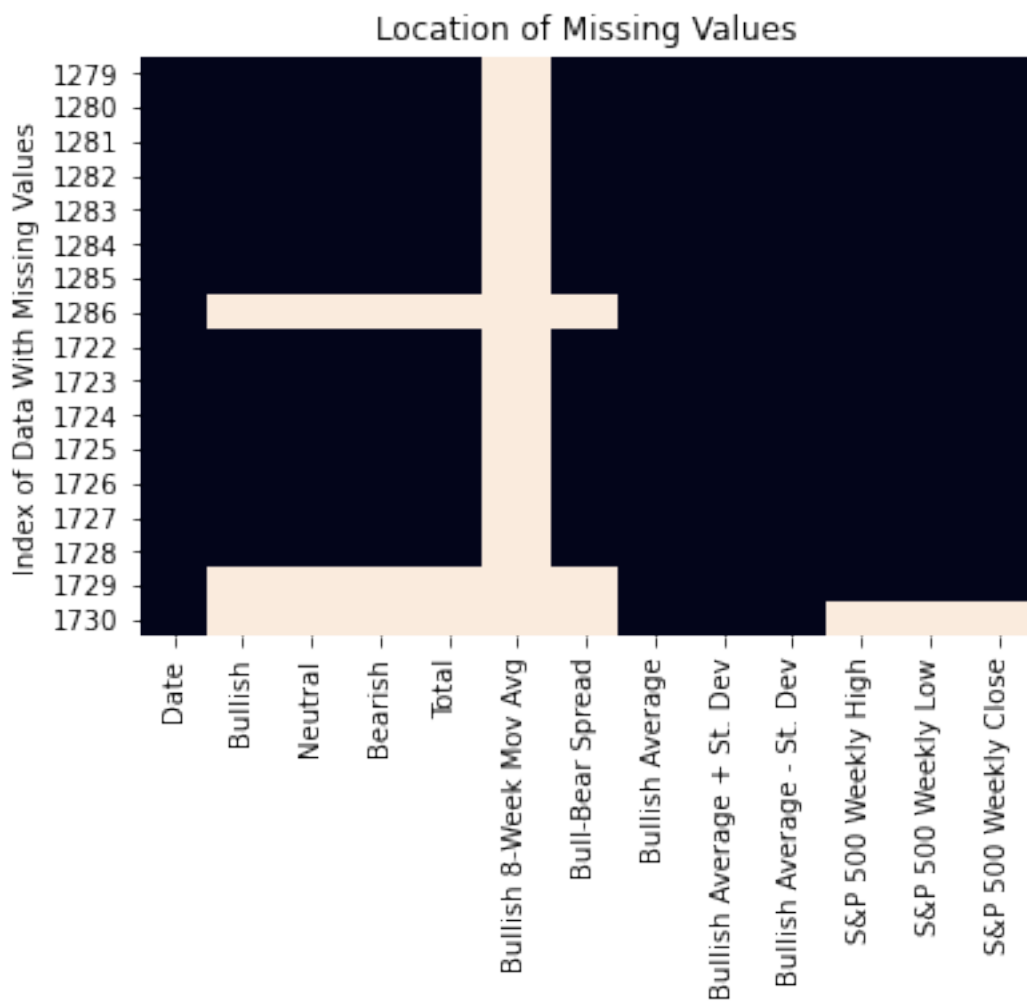
	Bull-Bear Spread	Bullish Average	Bullish Average + St. Dev	\
1279	0.18	0.386014	0.489178	
1280	0.20	0.386014	0.489178	
1281	0.29	0.386014	0.489178	
1282	0.37	0.386014	0.489178	
1283	0.39	0.386014	0.489178	
1284	0.12	0.386014	0.489178	
1285	0.13	0.386014	0.489178	
1286	NaN	0.386014	0.489178	
1722	0.01	0.386014	0.489178	
1723	0.22	0.386014	0.489178	
1724	0.60	0.386014	0.489178	
1725	0.25	0.386014	0.489178	
1726	0.27	0.386014	0.489178	
1727	0.00	0.386014	0.489178	
1728	0.22	0.386014	0.489178	
1729	NaN	0.386014	0.489178	
1730	NaN	0.386014	0.489178	

	Bullish Average - St. Dev	S&P 500 Weekly High	S&P 500 Weekly Low	\
1279	0.28285	659.08	635.00	
1280	0.28285	663.00	638.79	
1281	0.28285	664.23	646.99	
1282	0.28285	656.37	641.43	
1283	0.28285	639.26	621.42	
1284	0.28285	621.62	610.65	
1285	0.28285	612.92	598.47	
1286	0.28285	619.15	597.29	

1722	0.28285	329.79	316.70
1723	0.28285	331.38	323.04
1724	0.28285	335.90	329.25
1725	0.28285	334.65	323.00
1726	0.28285	323.00	316.23
1727	0.28285	318.66	310.65
1728	0.28285	311.39	307.81
1729	0.28285	314.59	307.63
1730	0.28285	NaN	NaN

	S&P 500 Weekly Close
1279	644.37
1280	659.08
1281	647.98
1282	656.37
1283	635.84
1284	621.62
1285	611.83
1286	601.81
1722	316.70
1723	323.04
1724	335.90
1725	333.99
1726	323.00
1727	318.66
1728	309.27
1729	314.59
1730	NaN

```
[4]: ax = sns.heatmap(nadata.isna(), cbar=False)
      ax.set_title('Location of Missing Values')
      ax.set_ylabel('Index of Data With Missing Values')
      plt.show()
```



Here are data from the oldest few dates in the data set:

```
[5]: fulldata.tail(15)
```

```
[5]:
```

	Date	Bullish	Neutral	Bearish	Total	Bullish 8-Week Mov Avg \
1716	1987-10-16	0.37	0.30	0.33	1.0	0.48000
1717	1987-10-09	0.46	0.39	0.15	1.0	0.51625
1718	1987-10-02	0.44	0.26	0.30	1.0	0.51500
1719	1987-09-25	0.53	0.17	0.30	1.0	0.53000
1720	1987-09-18	0.60	0.29	0.11	1.0	0.49625
1721	1987-09-11	0.50	0.23	0.27	1.0	0.46625
1722	1987-09-04	0.42	0.17	0.41	1.0	NaN
1723	1987-08-28	0.52	0.18	0.30	1.0	NaN
1724	1987-08-21	0.66	0.28	0.06	1.0	NaN
1725	1987-08-14	0.45	0.35	0.20	1.0	NaN
1726	1987-08-07	0.56	0.15	0.29	1.0	NaN

1727	1987-07-31	0.26	0.48	0.26	1.0	NaN
1728	1987-07-24	0.36	0.50	0.14	1.0	NaN
1729	1987-07-17	NaN	NaN	NaN	NaN	NaN
1730	1987-06-26	NaN	NaN	NaN	NaN	NaN

	Bull-Bear Spread	Bullish Average	Bullish Average + St. Dev	\
1716	0.04	0.384907	0.488293	
1717	0.31	0.384907	0.488293	
1718	0.14	0.384907	0.488293	
1719	0.23	0.384907	0.488293	
1720	0.49	0.384907	0.488293	
1721	0.23	0.384907	0.488293	
1722	0.01	0.386014	0.489178	
1723	0.22	0.386014	0.489178	
1724	0.60	0.386014	0.489178	
1725	0.25	0.386014	0.489178	
1726	0.27	0.386014	0.489178	
1727	0.00	0.386014	0.489178	
1728	0.22	0.386014	0.489178	
1729	NaN	0.386014	0.489178	
1730	NaN	0.386014	0.489178	

	Bullish Average - St. Dev	S&P 500 Weekly High	S&P 500 Weekly Low	\
1716	0.281522	314.52	282.70	
1717	0.281522	328.08	311.07	
1718	0.281522	328.07	321.69	
1719	0.281522	321.19	310.54	
1720	0.281522	323.08	314.86	
1721	0.281522	321.98	313.56	
1722	0.282850	329.79	316.70	
1723	0.282850	331.38	323.04	
1724	0.282850	335.90	329.25	
1725	0.282850	334.65	323.00	
1726	0.282850	323.00	316.23	
1727	0.282850	318.66	310.65	
1728	0.282850	311.39	307.81	
1729	0.282850	314.59	307.63	
1730	0.282850	NaN	NaN	

	S&P 500 Weekly Close
1716	282.70
1717	311.07
1718	328.07
1719	320.16
1720	314.86
1721	321.98
1722	316.70

1723	323.04
1724	335.90
1725	333.99
1726	323.00
1727	318.66
1728	309.27
1729	314.59
1730	NaN

Observations made from the above tables: 1. Most of the missing values appear in the Bullish 8-Week Mov Avg column. Since it takes average of the Bullish column over an 8-week window, it is supposed to have a value only after 7 dates with available Bullish data, which brings us to 1987-9-11 and after 1. The earliest two dates do not even have Bullish Neutral and Bearish. They can be removed from the data set 1. For some reason Bullish, Neutral and Bearish data are also missing on 1996-1-11 which results in missing values in the Bullish 8-Week Mov Avg column on 8 dates (1996-1-11 and the following 7 weeks)

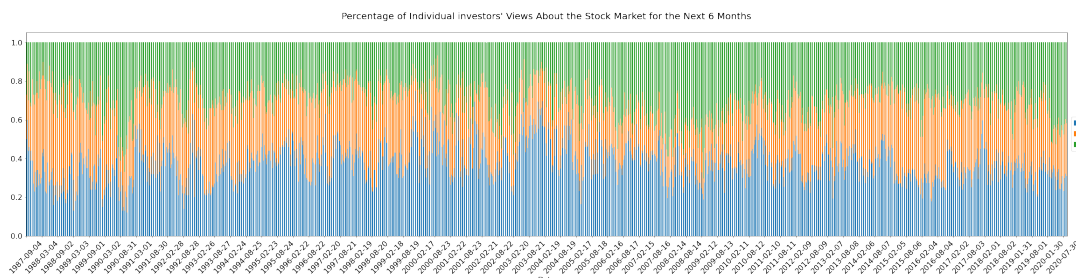
Question Three: Create a figure that shows how the proportion of investors who are bullish/neutral/bearish changes over the **years** of the data set.

Answer:

```
[6]: tick_spacing = 26

plt.rc('xtick',labelsize=16)
plt.rc('ytick',labelsize=16)
params = {'legend.fontsize': 16,
          'legend.handlelength': 2}
plt.rcParams.update(params)

ax = fulldata[:-8][::-1].set_index('Date')[['Bullish', 'Neutral', 'Bearish']].
    ↪plot(kind='bar', stacked=True, figsize=[40, 8], rot=45)
ax.set_title("Percentage of Individual investors' Views About the Stock Market_
    ↪for the Next 6 Months\n", fontsize=20)
ax.set_xticks(ax.get_xticks()[::tick_spacing])
ax.set_xlabel('Date', fontsize=16)
plt.legend(loc='center left', bbox_to_anchor=(1.0, 0.5))
plt.show()
```



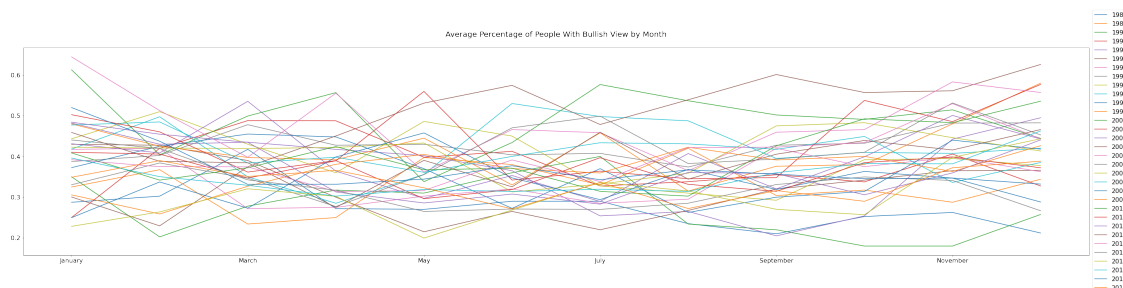
Question Four: A colleague wonders if there could be a seasonal effect, i.e., that there are natural variations in investor sentiment over the course of a year that tend to repeat from year to year. Create a plot that explores whether or not there is evidence for this. Be sure to include a description of your conclusion. **Do not perform any “formal” analyses.**

Answer:

For each month in the data set, we take average of the percentage of people who have a bullish view. Then we get a line plot of the monthly average for each year. We find no obvious seasonal effect.

```
[7]: view = 'Bullish'

fulldata['Year'] = [date.year for date in fulldata['Date']]
fulldata['Month'] = [date.month for date in fulldata['Date']]
monthly_bullish_avg = fulldata[['Year', 'Month', view]].groupby(['Year', 'Month']).mean().unstack().dropna()
monthly_bullish_avg.columns = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']
ax = monthly_bullish_avg.T.plot(figsize=(50,10))
ax.set_title('Average Percentage of People With %s View by Month\n'%view, fontsize=20)
plt.legend(loc='center left', bbox_to_anchor=(1.0, 0.5))
plt.show()
```



But then we take month to month diff of the average and things will become a bit more interesting.

```
[8]: mon_to_mon_chg = fulldata[['Year', 'Month', view]].groupby(['Year', 'Month']).mean().diff().unstack().dropna()
mon_to_mon_chg.columns = monthly_bullish_avg.columns
mon_to_mon_chg
```

```
[8]:
```

	January	February	March	April	May	June \
Year						
1988	0.019500	0.015000	0.115000	-0.145500	-0.002000	0.020000
1989	0.113000	0.042500	-0.133500	0.016000	0.147500	-0.017500

1990	0.006000	-0.147500	0.075500	0.039500	-0.007500	0.050000
1991	-0.007500	0.170000	0.068000	-0.000500	-0.089500	0.014500
1992	0.154500	-0.052000	0.005000	-0.017500	-0.015500	-0.037000
1993	-0.065000	-0.002500	-0.037500	-0.090000	-0.085000	0.050000
1994	0.051500	0.002500	-0.148000	0.003000	0.027500	0.017500
1995	0.025000	0.015000	0.075500	-0.050500	0.002500	-0.026000
1996	-0.039167	0.066667	-0.080000	-0.070000	0.126000	-0.036000
1997	0.040500	0.079500	-0.147500	-0.065000	0.071000	0.174000
1998	-0.047500	0.052500	0.030000	-0.007000	-0.078000	-0.097500
1999	0.062000	-0.055000	-0.027500	-0.005500	0.053000	-0.115000
2000	0.032750	-0.208700	0.095150	0.057550	-0.218700	0.096025
2001	-0.034175	-0.004525	-0.058735	0.042310	0.170350	-0.217775
2002	-0.145645	-0.022530	0.126775	-0.171525	-0.066840	0.056165
2003	-0.142490	-0.070885	0.145175	0.075525	0.080685	0.043640
2004	0.018625	-0.131900	-0.114600	0.157400	-0.193775	0.104400
2005	-0.226610	0.053550	-0.037400	-0.069575	0.078350	0.114865
2006	-0.030140	-0.002000	-0.001380	0.002855	0.010875	-0.120230
2007	0.063800	0.007550	-0.100765	0.014265	-0.037225	0.039175
2008	-0.135580	0.087605	-0.064300	0.125725	0.059090	-0.110440
2009	-0.026505	-0.046920	0.068175	0.053965	0.024135	-0.034625
2010	-0.017860	-0.066800	0.030475	0.051965	-0.057090	0.004050
2011	-0.033135	-0.042150	-0.098775	0.028150	-0.093425	0.022415
2012	0.111889	-0.029491	-0.019924	-0.121869	-0.027105	0.021884
2013	0.018328	-0.070980	-0.003456	-0.109683	0.115559	-0.064976
2014	-0.074046	-0.017169	-0.000352	-0.060600	0.002390	0.075198
2015	-0.003103	-0.020611	-0.077247	-0.025531	-0.051905	0.005817
2016	-0.038269	0.035484	0.056735	-0.018558	-0.102458	0.068887
2017	-0.047400	-0.045677	-0.021235	-0.026576	0.016216	-0.004799
2018	0.057434	-0.082560	-0.106027	0.000292	0.012634	0.032873
2019	0.060712	0.040959	-0.039044	0.015211	-0.043059	-0.051767

	July	August	September	October	November	December
Year						
1988	-5.551115e-17	-0.055000	-0.025000	0.042500	0.010000	-0.050500
1989	-5.000000e-02	0.090000	-0.116000	0.013500	-0.030000	0.056500
1990	4.000000e-02	-0.166000	-0.014000	-0.040000	0.000000	0.077500
1991	-8.500000e-02	0.010500	0.017000	-0.017500	0.068500	-0.078500
1992	-1.110000e-01	0.011000	-0.060000	0.049000	0.188500	0.052500
1993	-4.500000e-02	0.047500	0.052500	0.022000	0.023000	0.001000
1994	-3.500000e-02	0.010000	0.125000	-0.045000	0.023000	-0.035500
1995	3.500000e-03	-0.033500	0.053500	0.005000	0.049500	0.000500
1996	-1.225000e-01	-0.017500	-0.017500	0.099500	0.003000	-0.017500
1997	-3.200000e-02	-0.010500	-0.092500	0.015000	-0.002500	0.012500
1998	9.750000e-02	-0.107500	0.037500	0.014000	0.126000	-0.022000
1999	1.280000e-01	-0.143000	0.061000	0.004000	0.100000	0.100000
2000	1.425000e-01	-0.039875	-0.034800	-0.010925	0.023485	-0.069985
2001	2.190000e-02	-0.018565	0.002740	0.189575	-0.052115	0.091365

2002	-6.380000e-02	0.117755	-0.086855	0.078835	0.101515	-0.057725
2003	-9.616000e-02	0.060735	0.062000	-0.044155	0.004280	0.064575
2004	-7.725000e-03	-0.093375	0.094535	0.006615	0.116975	-0.025990
2005	2.876200e-02	-0.116502	0.011200	0.016600	0.121025	-0.078485
2006	1.950500e-02	0.026675	0.116025	0.007800	-0.036685	-0.032765
2007	3.347500e-02	-0.002770	-0.013405	0.031675	-0.113030	0.049180
2008	-5.356000e-02	0.074285	-0.049225	0.044340	-0.016290	-0.014825
2009	-1.095000e-02	0.063225	-0.031300	0.008815	-0.037115	0.063435
2010	-5.690000e-02	-0.011800	0.123620	0.066330	-0.009475	0.052685
2011	7.821000e-02	-0.065475	-0.017115	0.070590	0.013925	-0.026645
2012	-2.468825e-02	0.077956	-0.003088	-0.051938	0.053747	0.081044
2013	1.340175e-01	-0.115216	0.061291	0.033694	-0.023287	0.050113
2014	-5.294745e-02	0.083299	0.001005	0.011201	0.095674	-0.086271
2015	-9.024500e-04	0.015275	0.045820	0.018692	-0.008734	-0.074367
2016	6.803960e-02	-0.021325	-0.044951	-0.013320	0.140779	0.045779
2017	5.415000e-03	-0.005301	0.045985	0.022255	-0.000449	0.080657
2018	-3.446875e-02	0.024476	-0.010767	-0.011005	0.001443	-0.058725
2019	6.493225e-02	-0.064022	0.044126	-0.026440	0.082511	0.016054

Below is a summary of the monthly average data.

```
[9]: mon_to_mon_chg.describe()
```

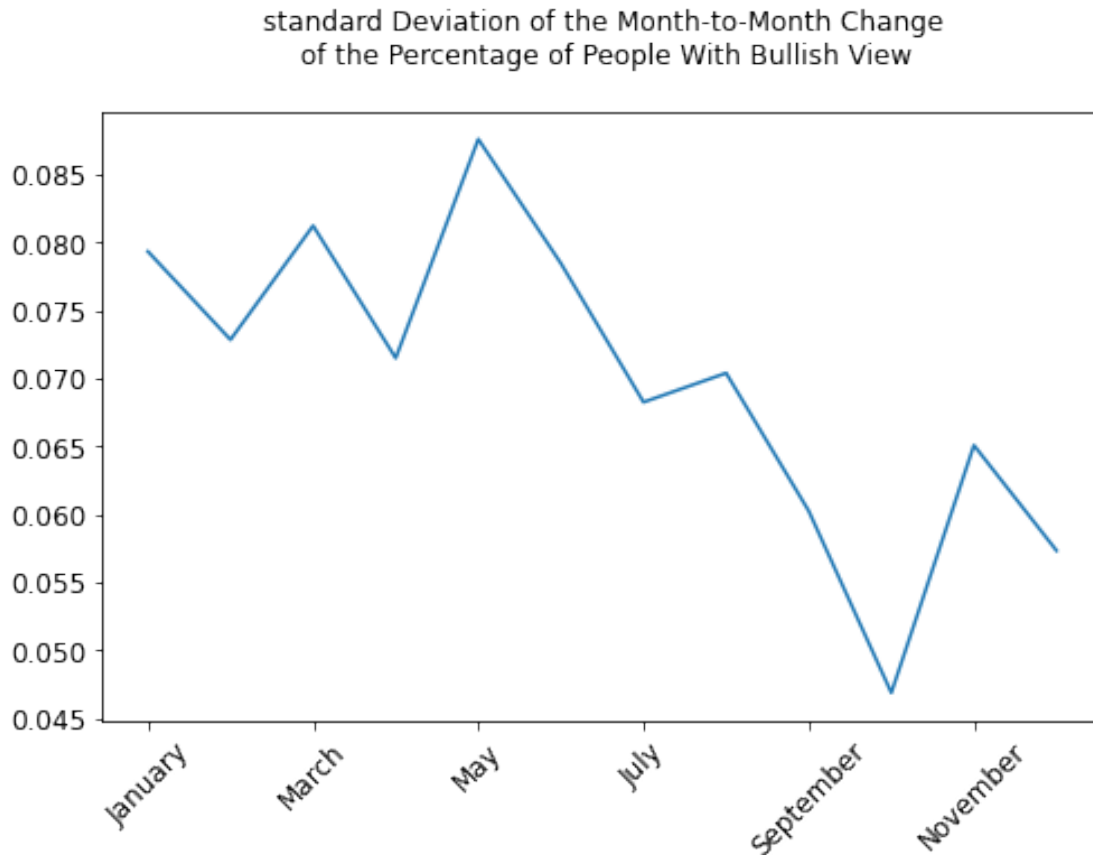
```
[9]:
```

	January	February	March	April	May	June \
count	32.000000	32.000000	32.000000	32.000000	32.000000	32.000000
mean	-0.008706	-0.014096	-0.013305	-0.011616	-0.005353	0.002431
std	0.080611	0.074017	0.082535	0.072644	0.089007	0.079718
min	-0.226610	-0.208700	-0.148000	-0.171525	-0.218700	-0.217775
25%	-0.041225	-0.052750	-0.077935	-0.061700	-0.059527	-0.036250
50%	-0.005302	-0.010847	-0.020579	-0.003000	0.000195	0.016000
75%	0.043250	0.036853	0.059551	0.030987	0.054523	0.050000
max	0.154500	0.170000	0.145175	0.157400	0.170350	0.174000

	July	August	September	October	November	December
count	32.000000	32.000000	32.000000	32.000000	32.000000	32.000000
mean	-0.000526	-0.011704	0.010573	0.019101	0.031787	0.005175
std	0.069350	0.071522	0.061217	0.047629	0.066126	0.058249
min	-0.122500	-0.166000	-0.116000	-0.051938	-0.113030	-0.086271
25%	-0.050737	-0.057255	-0.026575	-0.010945	-0.008919	-0.039250
50%	-0.004314	-0.007901	0.001873	0.013750	0.011962	0.000750
75%	0.035106	0.031881	0.052750	0.035895	0.085802	0.053639
max	0.142500	0.117755	0.125000	0.189575	0.188500	0.100000

The standard deviation of the data seems to change more in the first half of a year, and to change less in the second half. The standard deviation reaches its minimum in October. Since we are looking at month to month changes, this means the percentage of people with a bullish view in September and the same percentage in October tend not to change as much as early months.

```
[10]: plt.rc('xtick',labelsize=12)
plt.rc('ytick',labelsize=12)
ax = mon_to_mon_chg.apply(np.std).plot(rot=45, figsize=(8, 5))
ax.set_title('standard Deviation of the Month-to-Month Change\n of the_
↳Percentage of People With %s View\n'%view)
plt.show()
```



Next we explore the pair plot. To determine the x limit and y limit we find the maximum and minimum of the monthly average data.

```
[11]: mon_to_mon_chg.values.min(), mon_to_mon_chg.values.max()
```

```
[11]: (-0.22661000000000001, 0.18957500000000005)
```

The aforementioned volatility structure is also obvious in the pair plots. But aside from that there does not seem to have other interesting patterns.

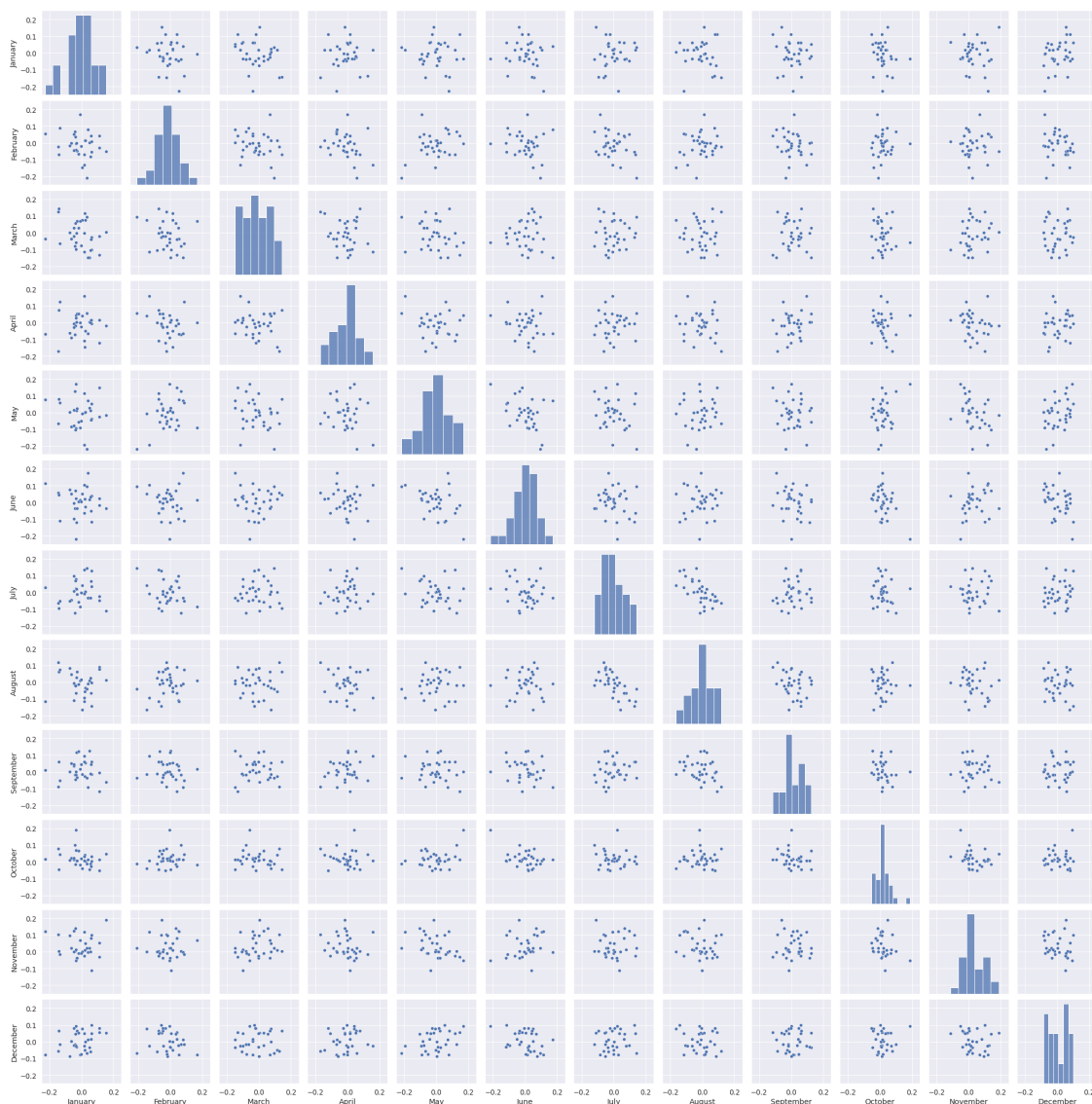
```
[12]: sns.set(font_scale=1.2)
g = sns.pairplot(mon_to_mon_chg)
for row in g.axes:
```

```

for ax in row:
    ax.set_xlim((-0.25, 0.25))
    ax.set_ylim((-0.25, 0.25))

plt.show()

```



The conclusion so far, as mentioned previously, is that the percentage of people with a bullish view in September and the same percentage in October tend not to change as much as early months in a year, although that does not mean people tend not to change their view from September to October. For example, a survey conducted on 100 people in September could show 50 bullish and 50 bearish, but in October every one changes their view and as a result the percentage stays the same. It could still be the case that “people tend not to change their views in fall”. Without more detailed data, it is just hard to further explore.

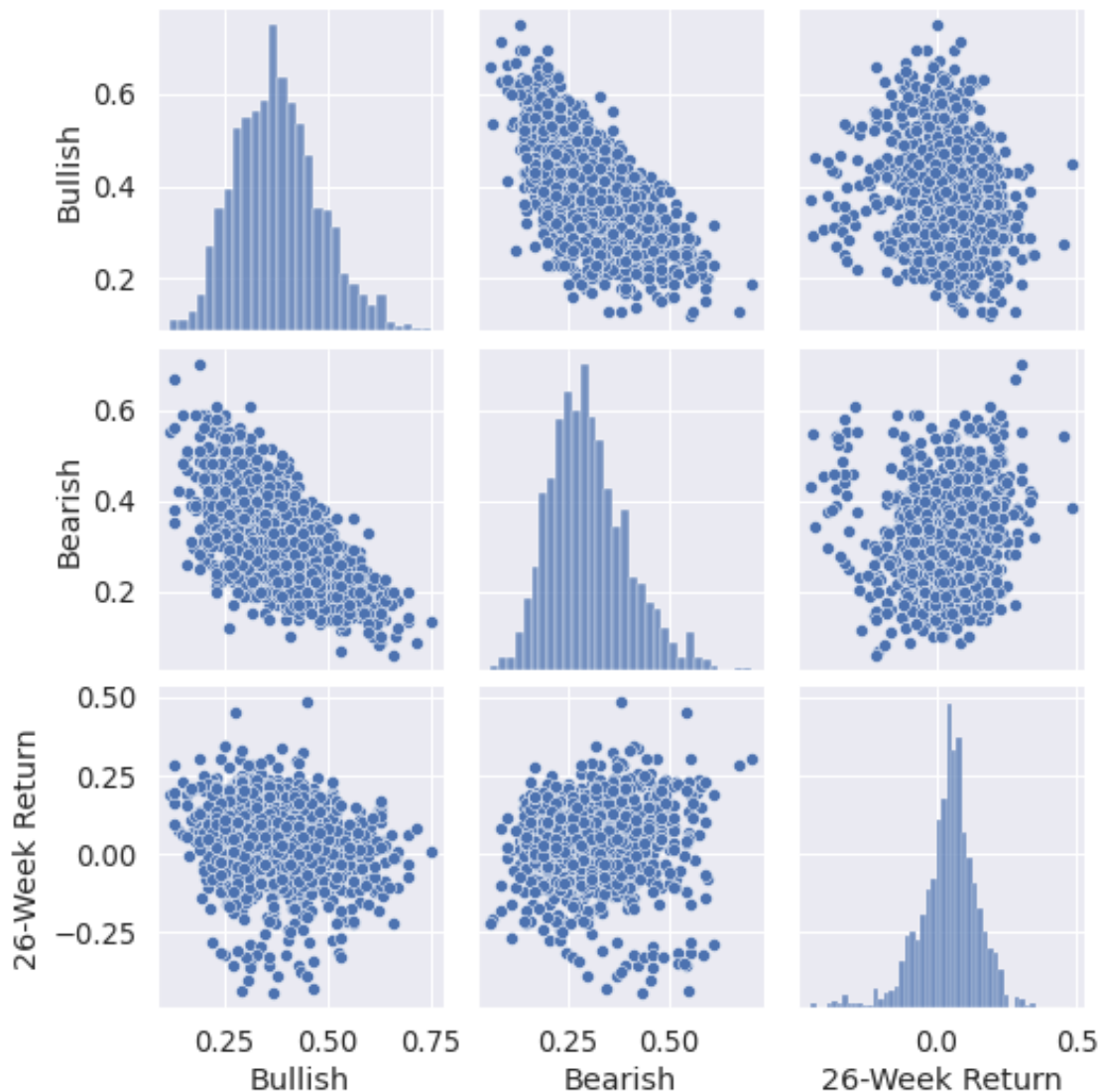
If we do the same for Bearish view percentages, the result will be very similar. In the early months of a year, the percentage tends to change more while in the later months it changes less, with the September standard deviation being the smallest and the October one the second smallest. To see the results on the Bearish view, simply go back to the first code cell in this question, set `view = 'Bearish'`, and rerun the notebook.

Question Five: Do you think that investors are generally “correct” in their sentiments? In other words, does their opinion regarding the future of the market predict where the market will be over the following six months? Create a plot that explores this. Be sure to include a description of your conclusion. **Do not perform any “formal” analyses.**

Answer:

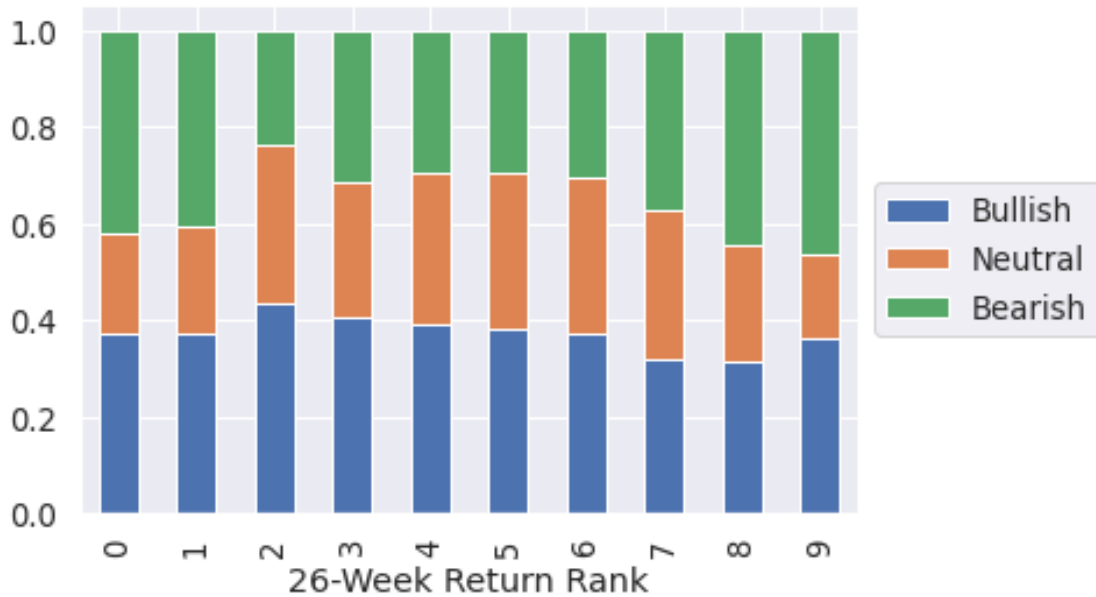
First we construct a 26-Week Return column in the data, representing the return of the following 26 weeks (forward looking). Simple pair plots on the return and the Bullish and Bearish percentages show no evidence of any predictive power the survey results might have.

```
[50]: fulldata['Close in 26 Weeks'] = fulldata['S&P 500 Weekly Close'].shift(26)
      fulldata['Previous Weekly Close'] = fulldata['S&P 500 Weekly Close'].shift(-1)
      fulldata['26-Week Return'] = fulldata['Close in 26 Weeks']/fulldata['Previous_
      ↪Weekly Close'] - 1
      sns.pairplot(fulldata[['Bullish', 'Bearish', '26-Week Return']].dropna())
      plt.show()
```



Next we divide the data set by the 26-Week Return into 10 categories, 0 being the worst (negative) return and 9 being the best. In each group we take average of the Bullish, Neutral, and Bearish percentages. The result is presented in a stacked bar chart. The result shows that, in the beginning of a period of 6 months where the market has the best return (rank 9), the bearish percentage is actually the highest. In contrast, the rank 2 bin has the lowest bearish percentage. In fact, from rank 2 to rank 8 the bearish percentage is roughly increasing while the bullish percentage is decreasing. Intuitively, the result says the more bearish the investors are, the more you should buy (and hold 6 months). Rank 0, 1 and 9 do not follow this rule, likely because the return is so extreme there are not enough data points to make meaningful inferences (like the average). This observation could be used to fit a Gaussian mixture model to determine if one should buy and hold 6 month at any point in time.

```
[54]: fulldata['26-Week Return Rank'] = pd.cut(fulldata['26-Week Return'], 10,
        ↳labels=range(10))
fulldata[['26-Week Return Rank', 'Bullish', 'Neutral', 'Bearish']].dropna().
        ↳groupby('26-Week Return Rank').mean().plot(kind='bar', stacked=True, )
plt.legend(loc='center left', bbox_to_anchor=(1.0, 0.5))
plt.show()
```



Below we also present the number of days in each category, and the actual return range of each categories.

```
[55]: fulldata[['26-Week Return Rank', 'Bullish']].dropna().groupby('26-Week Return_
        ↳Rank').count().rename(columns={'Bullish': 'Count'})
```

```
[55]:
```

	Count
26-Week Return Rank	
0	11
1	20
2	31
3	143
4	363
5	740
6	309
7	73
8	10
9	2

```
[58]: pd.cut(fulldata['26-Week Return'], 10)
```

```

[58]: 0          NaN
      1          NaN
      2          NaN
      3          NaN
      4          NaN
      ...
      1726      (-0.259, -0.166]
      1727      (-0.259, -0.166]
      1728      (-0.259, -0.166]
      1729          NaN
      1730          NaN
Name: 26-Week Return, Length: 1731, dtype: category
Categories (10, interval[float64]): [(-0.445, -0.351] < (-0.351, -0.259] <
(-0.259, -0.166] < (-0.166, -0.0736] ... (0.111, 0.204] < (0.204, 0.297] <
(0.297, 0.389] < (0.389, 0.482]]

```

chunyuac_HW3

October 22, 2020

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

1.

Let m_1 and m_2 be the sample first and second moments, respectively, that is

$$m_1 = \sum_{i=1}^n X_i/n,$$
$$m_2 = \sum_{i=1}^n X_i^2/n.$$

Let $X \sim \text{Gamma}(\alpha, \beta)$. From the Table of Distribution Facts we know $E(X) = \alpha/\beta$, $V(X) = E(X^2) - E(X)^2 = \alpha/\beta^2$ and hence

$$E(X^2) = \frac{\alpha^2 + \alpha}{\beta^2}.$$

Now matching the first two moments we get

$$m_1 = \frac{\alpha}{\beta},$$
$$m_2 = \frac{\alpha^2 + \alpha}{\beta^2}.$$

and thus $m_2 - m_1^2 = \alpha/\beta^2$. So

$$\beta = \frac{\frac{\alpha}{\beta}}{\frac{\alpha}{\beta^2}} = \frac{m_1}{m_2 - m_1^2},$$

which is the desired estimator for β . Plug this back in to $m_1 = \frac{\alpha}{\beta}$ to obtain the estimator for α

$$\alpha = \frac{m_1^2}{m_2 - m_1^2}.$$

2.

(a)

Since $n \geq 30$, the 95% confidence interval is

$$\left[\bar{x} - z_{.025} \left(\frac{s}{\sqrt{n}} \right), \bar{x} + z_{.025} \left(\frac{s}{\sqrt{n}} \right) \right],$$

where $z_{.025}$ is such that $P(Z > z_{.025}) = 2.5\%$ when Z has the standard normal distribution, whose value can be found to be -1.96 in python as follows:

```
[1]: from scipy.stats import norm  
  
norm.ppf(1 - 0.025)
```

```
[1]: 1.959963984540054
```

Thus, numerically the confidence interval can be evaluated as follows:

```
[2]: import numpy as np  
  
x = 14.3  
s = 4.2  
n = 40  
z = norm.ppf(1 - 0.025)  
  
(x-z*s/np.sqrt(n), x+z*s/np.sqrt(n))
```

```
[2]: (12.998430432160422, 15.60156956783958)
```

(b)

Since n is small and the population distribution is unknown, none of the confidence intervals covered in the lecture is applicable. If σ were known, there is Chebyshev's inequality to guarantee

$$P\left(\bar{x} - k \frac{\sigma}{\sqrt{n}} < \mu < \bar{x} + k \frac{\sigma}{\sqrt{n}}\right) > 1 - \frac{1}{k^2}.$$

Setting $k = \sqrt{20}$ leads to a 95% confidence interval. But here σ is also unknown. The only clue is the sample standard deviation $s = 4.2$, which is not guaranteed to be close to σ either. There might still be a rigorous way to construct a 95% confidence interval but I feel like even if there is one, the constructed confidence interval will not be meaningful, maybe some wide range unless n is large, but if n is large we can just apply CLT.

(c)

Since $n < 30$ but the population distribution is known to be normal, the 95% confidence interval is

$$\left[\bar{x} - t_{.025,9} \left(\frac{s}{\sqrt{n}} \right), \bar{x} + t_{.025,9} \left(\frac{s}{\sqrt{n}} \right) \right],$$

where $t_{.025,9}$ is such that $P(T > t_{.025,9}) = 2.5\%$ when T has the t -distribution with 9 degrees of freedom, whose value can be found to be 2.262 in python as follows:

```
[3]: from scipy.stats import t  
  
t.ppf(1-0.025, 9)
```

```
[3]: 2.2621571627409915
```

Thus, numerically the confidence interval can be evaluated as follows:

```
[4]: x = 14.3  
s = 4.2  
n = 10  
p = t.ppf(1-0.025, 9)  
  
(x-p*s/np.sqrt(n), x+p*s/np.sqrt(n))
```

```
[4]: (11.295500994999198, 17.304499005000803)
```

3.

(a)

The log likelihood function is

$$l(\lambda) = \log \left(\prod_{i=1}^n \frac{e^{-\lambda} \lambda^{X_i}}{X_i!} \right) = \sum_{i=1}^n (-\lambda + X_i \log(\lambda) - \log(X_i!)).$$

Take the first derivative with respect to λ and set to zero to get

$$\frac{dl}{d\lambda} = \sum_{i=1}^n \left(-1 + \frac{X_i}{\lambda} \right) = -n + \frac{\sum_{i=1}^n X_i}{\lambda} = 0$$

and hence $\lambda = \sum_{i=1}^n X_i / n = \bar{X}$. Now check the second derivative

$$\frac{d^2 l}{d\lambda^2} = -\frac{\sum_{i=1}^n X_i}{\lambda^2} \leq 0.$$

Thus we can conclude that $\bar{X}$ is indeed the maximum likelihood estimator for λ .

(b)

Since $E(\bar{X}) = \sum_{i=1}^n E(X_i)/n = \lambda$, we know $\bar{X}$ is an unbiased estimator of λ .

(c)

The standard error can be found as

$$\sqrt{V(\bar{X})} = \sqrt{V\left(\frac{\sum_{i=1}^n X_i}{n}\right)} = \sqrt{\frac{\sum_{i=1}^n V(X_i)}{n^2}} = \sqrt{\frac{V(X_i)}{n}} = \sqrt{\frac{\lambda}{n}}.$$

4.

Scipy only generates exponential random numbers with parameter 1. To generate exponential random numbers with parameter λ we use this property: If $X \sim \text{Exponential}(1)$ then $X/\lambda \sim \text{Exponential}(\lambda)$.

As the result shown below, only 55% of the time the adjusted estimator actually generates a better estimate than the “worst” estimator does.

```
[5]: from scipy.stats import expon

n = 20
l = 10

numRepetition = 100000
numAdjBetter = 0

for _ in range(numRepetition):
    sample = expon.rvs(size=n)/l

    lMom2 = np.sqrt(2*n/((sample**2).sum()))
    lMom1 = 1/sample.mean()
    lAdj = lMom1*(n-1)/n

    if abs(lAdj - l) < abs(lMom2 - l):
        numAdjBetter += 1

numAdjBetter/numRepetition
```

```
[5]: 0.55311
```

5.

(a)

Since $E(\hat{\theta}_1) = \frac{4}{3}E(\bar{X}) = \frac{4}{3}E(X) = \theta$, we know that $\hat{\theta}_1$ is an unbiased estimator for θ .

(b)

We have

$$\begin{aligned}\text{bias}(\hat{\theta}_2) &= E(\hat{\theta}_2) - \theta \\ &= \int_0^\theta x f_{X_{(n)}}(x) dx - \theta \\ &= \int_0^\theta \frac{3nx^{3n}}{\theta^{3n}} dx - \theta \\ &= \frac{3n\theta^{3n+1}}{(3n+1)\theta^{3n}} - \theta \\ &= -\frac{\theta}{3n+1} \neq 0.\end{aligned}$$

Thus $\hat{\theta}_2$ is biased.

(c)

First we find MSE for both estimators:

$$\begin{aligned}\text{MSE}(\hat{\theta}_2) &= V(\hat{\theta}_2) + \text{bias}^2(\hat{\theta}_2) \\ &= \left(\frac{3n}{(3n+2)(3n+1)^2} + \frac{1}{(3n+1)^2} \right) \theta^2 \\ &= \frac{2\theta^2}{(3n+1)(3n+2)}, \\ \text{MSE}(\hat{\theta}_1) &= V(\hat{\theta}_1) = \frac{16}{9}V(\bar{X}) = \frac{16}{9n} \frac{3}{80} \theta^2 = \frac{\theta^2}{15n}.\end{aligned}$$

Thus $\text{MSE}(\hat{\theta}_2) < \text{MSE}(\hat{\theta}_1)$ if and only if

$$\frac{2}{(3n+1)(3n+2)} < \frac{1}{15n}.$$

Since $n > 0$, this is equivalent to $30n < (3n+1)(3n+2)$, which solves to $\{n > 2.23385\} \cup \{n < 0.09948\}$. Since n can only be integers, this is the same as $n > 2$.

chunyuac_HW4

October 21, 2020

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

1.

(a)

The likelihood function is

$$L(\alpha) = \prod_{i=1}^n \frac{\alpha \lambda^\alpha}{X_i^{\alpha+1}},$$

so the log likelihood is

$$l(\alpha) = \sum_{i=1}^n (\log \alpha + \alpha \log \lambda - (\alpha + 1) \log X_i)$$

and its derivative with respect to α

$$\begin{aligned} \frac{dl}{d\alpha} &= \sum_{i=1}^n \left(\frac{1}{\alpha} + \log \lambda - \log X_i \right) \\ &= \frac{n}{\alpha} + n \log \lambda - \sum_{i=1}^n \log X_i. \end{aligned}$$

Setting to zero and solve for α , we obtain the MLE

$$\hat{a} = \frac{1}{\frac{1}{n} \sum_{i=1}^n \log X_i - \log \lambda}.$$

(b)

The Fisher Information is

$$\begin{aligned} I(\alpha) &= E \left(-\frac{\partial^2 \log f(X; \alpha)}{\partial \alpha^2} \right) \\ &= E \left(-\frac{\partial^2}{\partial \alpha^2} (\log \alpha + \alpha \log \lambda - (\alpha + 1) \log X) \right) \\ &= E \left(-\frac{\partial}{\partial \alpha} \left(\frac{1}{\alpha} + \log \lambda - \log X \right) \right) \\ &= E \left(\frac{1}{\alpha^2} \right) = \frac{1}{\alpha^2}, \end{aligned}$$

so for large n , we have the approximation

$$\hat{\alpha} \approx N \left(\alpha, \frac{1}{nI(\alpha)} \right) = N(\alpha, \alpha^2/n).$$

In practice when α is unknown, we can replace α by an estimate given by the MLE $\hat{\alpha}$.

(c)

The distribution of α is approximately $N(\hat{\alpha}, \hat{\alpha}^2/n)$. Now given $\lambda = 2$ and a sample of size $n = 100$ such that $\sum_i \log(x_i) = 107.39$, we have an estimate

$$\hat{\alpha} = \frac{1}{\frac{1}{n} \sum_{i=1}^n \log x_i - \log \lambda} = \frac{1}{\frac{107.39}{100} - \log 2} = 2.626.$$

```
[1]: from math import log
1/(1.0739 - log(2))
```

```
[1]: 2.626375824269999
```

Thus the distribution of the MLE $\hat{\alpha}$ can be approximated by

$$N(\hat{\alpha}, \hat{\alpha}^2/n) = N(2.626, 0.2626^2).$$

The MLE $\hat{\alpha}$ is the best estimator for α in the sense that it is asymptotically efficient, meaning its bias vanishes as n goes to infinity (i.e., asymptotic unbiasedness), and among all asymptotically unbiased estimators, it achieves the lowest possible variance. Given the sample data and the above approximation, our best estimate for α is 2.626 and the standard error of the estimator $\hat{\alpha}$ is approximately 0.2626.

2.

(a)

The pdf is

$$f(x; \theta) = \frac{\beta^\theta}{\Gamma(\theta)} x^{\theta-1} e^{-\beta x}, \quad 0 < x < 1, \theta, \beta > 0,$$

so the Fisher Information is

$$\begin{aligned} I(\theta) &= E \left(-\frac{\partial^2 \log f(X; \theta)}{\partial \theta^2} \right) \\ &= E \left(-\frac{\partial^2}{\partial \theta^2} (\theta \log \beta + (\theta - 1) \log X - \log \Gamma(\theta) - \beta X) \right) \\ &= E \left(\frac{\partial}{\partial \theta} \frac{\Gamma'(\theta)}{\Gamma(\theta)} \right) \\ &= \frac{\Gamma(\theta) \Gamma''(\theta) - (\Gamma'(\theta))^2}{\Gamma^2(\theta)}. \end{aligned}$$

(b)

Let θ_0 be the true value of θ and $\hat{\theta}$ its MLE. When n is large, we have approximately

$$\begin{aligned} \hat{\theta} &\approx N \left(\theta_0, \frac{1}{nI(\theta_0)} \right) \\ &= N \left(\theta_0, \frac{\Gamma^2(\theta_0)}{n(\Gamma(\theta_0) \Gamma''(\theta_0) - (\Gamma'(\theta_0))^2)} \right). \end{aligned}$$

Since θ_0 is unknown, so is this distribution. It can be thought of as a distribution family. Given a sample data, in order to obtain a distribution we can work with, the best we can do is to numerically find an estimate using the MLE and replace all θ_0 by this estimate.

(c)

By the invariance property, the MLE for $\log(\theta)$ is $\log(\hat{\theta})$, whose distribution can be approximated by a normal using the Delta Method. Let

$$\sigma_n = \sqrt{\frac{\Gamma^2(\theta_0)}{n(\Gamma(\theta_0) \Gamma''(\theta_0) - (\Gamma'(\theta_0))^2)}}.$$

Recall that $\Gamma^2(\theta_0)/(\Gamma(\theta_0) \Gamma''(\theta_0) - (\Gamma'(\theta_0))^2)$ is the second derivative of $\log \Gamma(\theta)$ at θ_0 , which is finite given $\theta_0 > 0$. Thus we know that $\sigma_n \downarrow 0$ as $n \rightarrow \infty$. Since we have

$$\frac{\hat{\theta} - \theta_0}{\sigma_n} \xrightarrow{D} Z,$$

where $Z \sim N(0, 1)$, by the Delta Method with $g(x) = 1/x$, we know that $(g'(\theta_0))^2 = 1/\theta_0^2$ and

$$\log(\hat{\theta}) \approx N\left(\log(\theta_0), \frac{\Gamma^2(\theta_0)}{n\theta_0^2(\Gamma(\theta_0)\Gamma''(\theta_0) - (\Gamma'(\theta_0))^2)}\right).$$

Again here θ_0 is unknown. Given a sample data we can replace it by a maximum likelihood estimate to obtain a workable distribution.

3.

(a)

By the invariance property, the MLE for the odds is

$$\frac{\hat{p}}{1 - \hat{p}}.$$

(b)

Let $Y_1, Y_2, \dots, Y_n$ be i.i.d. from the Bernoulli(p) distribution. We can write $X = \sum_{i=1}^n Y_i$ and $\hat{p} = \bar{Y}$. We also have

$$\begin{aligned} E(\hat{p}) &= E(\bar{Y}) = E(Y_i) = p, \\ V(\hat{p}) &= V(\bar{Y}) = V\left(\sum_{i=1}^n Y_i/n\right) = V(Y_i)/n = p(1-p)/n. \end{aligned}$$

By the CLT, we know that

$$\frac{\hat{p} - p}{\sqrt{\frac{p(1-p)}{n}}} \xrightarrow{D} Z.$$

{-} We want to apply the Delta Method with $g(x) = x/(1-x)$, which requires $g'(p) \neq 0$ and $\sigma_n = \sqrt{p(1-p)/n} \downarrow 0$ as $n \rightarrow \infty$. This is true for all $0 < p < 1$. We know that

$$g'(p) = \frac{(1-p) + p}{(1-p)^2} = \frac{1}{(1-p)^2}$$

and that by the Delta Method

$$\begin{aligned} \frac{\hat{p}}{1 - \hat{p}} &\approx N(g(p), (g'(p))^2 \sigma_n^2) \\ &= N\left(\frac{p}{1-p}, \frac{1}{(1-p)^4} \frac{p(1-p)}{n}\right) \\ &= N\left(\frac{p}{1-p}, \frac{p}{n(1-p)^3}\right). \end{aligned}$$

To obtain a workable distribution, plug in $p = \hat{p}$.

(c)

Given the above approximation (the one before plugging in $p = \hat{p}$), we have

$$P\left(\left|\frac{\hat{p}}{1-\hat{p}} - \frac{p}{1-p}\right| < Z_{\alpha/2} \sqrt{\frac{p}{n(1-p)^3}}\right) = 1 - \alpha,$$

where $Z_{\alpha/2}$ is the $100(1 - \frac{\alpha}{2})$ percentile point of the standard normal distribution. Hence the confidence interval is

$$\frac{p}{1-p} \in \left[\frac{\hat{p}}{1-\hat{p}} - Z_{\alpha/2} \sqrt{\frac{p}{n(1-p)^3}}, \frac{\hat{p}}{1-\hat{p}} + Z_{\alpha/2} \sqrt{\frac{p}{n(1-p)^3}} \right]$$

Again this interval has an unknown p . To obtain a workable interval we plug in $p = \hat{p}$ and get

$$\left[\frac{\hat{p}}{1-\hat{p}} - Z_{\alpha/2} \sqrt{\frac{\hat{p}}{n(1-\hat{p})^3}}, \frac{\hat{p}}{1-\hat{p}} + Z_{\alpha/2} \sqrt{\frac{\hat{p}}{n(1-\hat{p})^3}} \right].$$

(d)

We look at the 95% confidence intervals. As the simulation results shown below, only around 65% of the 100,000 sample confidence intervals cover the true value of p , not 95%. The bar chart at the bottom shows the first 100 sample confidence intervals. The missing bars are intervals with length 0, which are simulation results with $\hat{p} = 0$. With $p = 0.1$ it is understandable that a sample value $\hat{p} = 0$ is not uncommon. Indeed, since the “rules of thumb” $np \geq 5, n(1-p) \geq 5$ are not satisfied, the CLT approximation applied in (b) is not expected to be accurate. So a simulation result like the below is to be expected.

```
[2]: from scipy.stats import norm, binom
import numpy as np
from pandas import DataFrame
import matplotlib.pyplot as plt

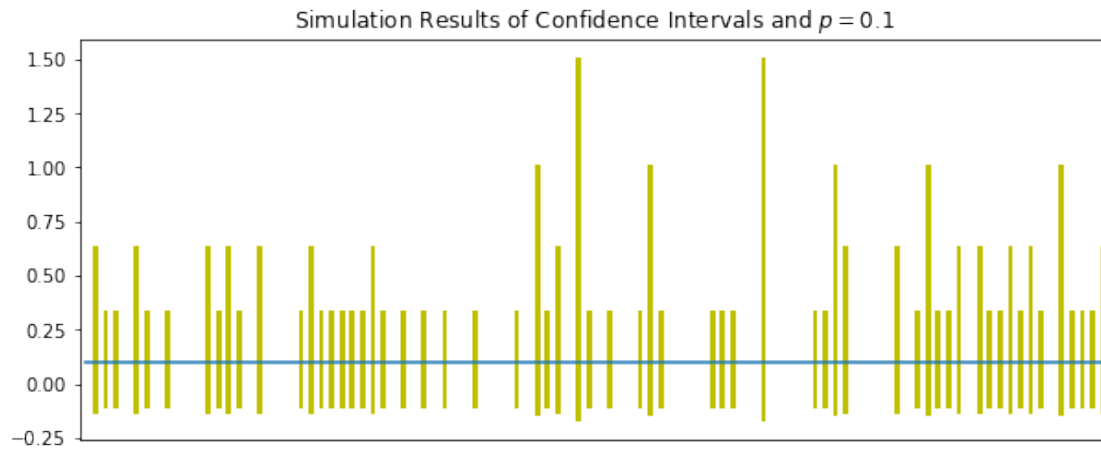
n = 10
p = 0.1
alpha = 5
sampleSize = 100000
z = norm.ppf(1 - alpha/100/2)

phat = binom.rvs(n, p, size=sampleSize)/n
l = phat/(1-phat) - z*np.sqrt(phat/(n*(1-phat)**3))
r = phat/(1-phat) + z*np.sqrt(phat/(n*(1-phat)**3))
p = np.full(sampleSize, p)

((l < p) & (p < r)).sum()/sampleSize
```

[2]: 0.65566

```
[3]: ax = DataFrame([l, r]).T.head(100).plot(kind='bar', stacked=True, legend=None,
      ↪color='y')
ax = DataFrame(p).head(100).plot(ax=ax, legend=None, figsize=(10, 4))
ax.set_xticks([])
ax.set_title("Simulation Results of Confidence Intervals and  $p=0.1$ ")
plt.show()
```



chunyuac_HW5

October 21, 2020

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

Question One

Reading data in and show the first few data points:

```
[1]: import pandas as pd

bonds = pd.read_csv('bonds.txt', sep='\t', index_col=0)
# bonds = bonds[bonds['CouponRate'] > 6]
n = len(bonds)

bonds.head()
```

```
[1]:      CouponRate  BidPrice
Case
1          7.000      92.94
2          9.000     101.44
3          7.000      92.66
4          4.125      94.50
5         13.125     118.94
```

There are 35 data points.

```
[2]: n
```

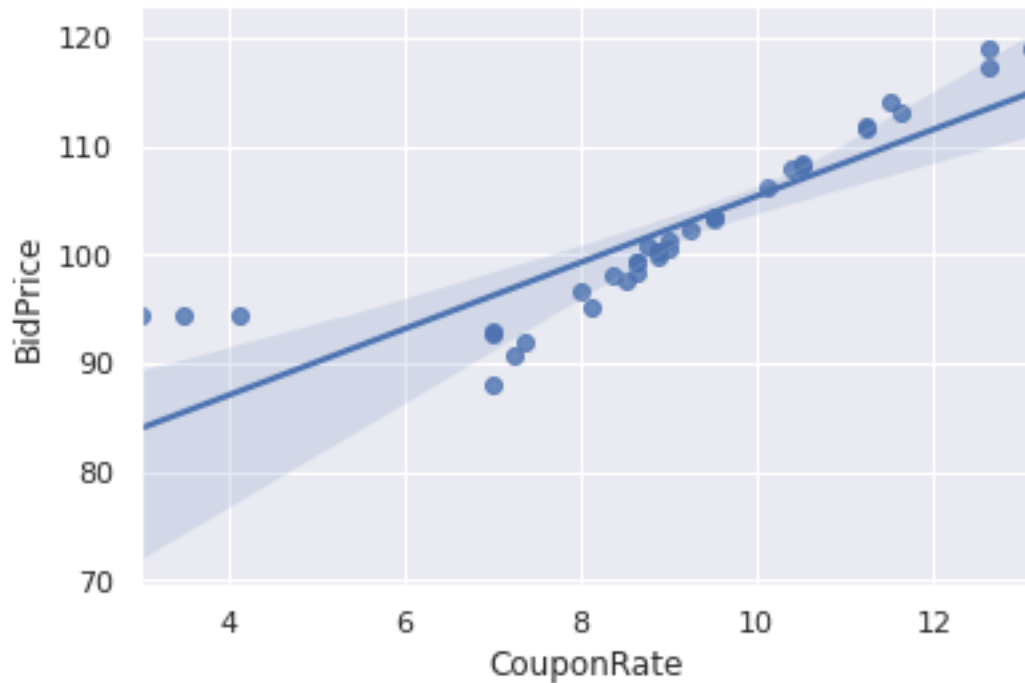
```
[2]: 35
```

1.

The below figure suggests that there might be outliers for low coupon Rates. Linear regression model does not fit the data very well.

```
[3]: import seaborn as sns
```

```
sns.set_theme(color_codes=True)
ax = sns.regplot(x='CouponRate', y='BidPrice', data=bonds)
```



2.

See below for the summary of the fitted model.

```
[4]: import statsmodels.api as sm
import matplotlib.pyplot as plt

olsBonds = sm.OLS.from_formula(formula='BidPrice ~ CouponRate', data=bonds).
    ↪fit()
olsBonds.summary()
```

```
[4]: <class 'statsmodels.iolib.summary.Summary'>
"""
```

```

                                OLS Regression Results
=====
Dep. Variable:                  BidPrice    R-squared:                   0.752
Model:                            OLS      Adj. R-squared:              0.744
Method:                 Least Squares    F-statistic:                   99.87
Date:                Thu, 15 Oct 2020    Prob (F-statistic):           1.64e-11
Time:                  09:01:31          Log-Likelihood:              -98.656
```

```

No. Observations:      35    AIC:      201.3
Df Residuals:          33    BIC:      204.4
Df Model:              1
Covariance Type:      nonrobust

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	74.7866	2.827	26.458	0.000	69.036	80.537
CouponRate	3.0661	0.307	9.994	0.000	2.442	3.690
Omnibus:		3.313	Durbin-Watson:			1.827
Prob(Omnibus):		0.191	Jarque-Bera (JB):			2.262
Skew:		0.607	Prob(JB):			0.323
Kurtosis:		3.276	Cond. No.			37.3

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

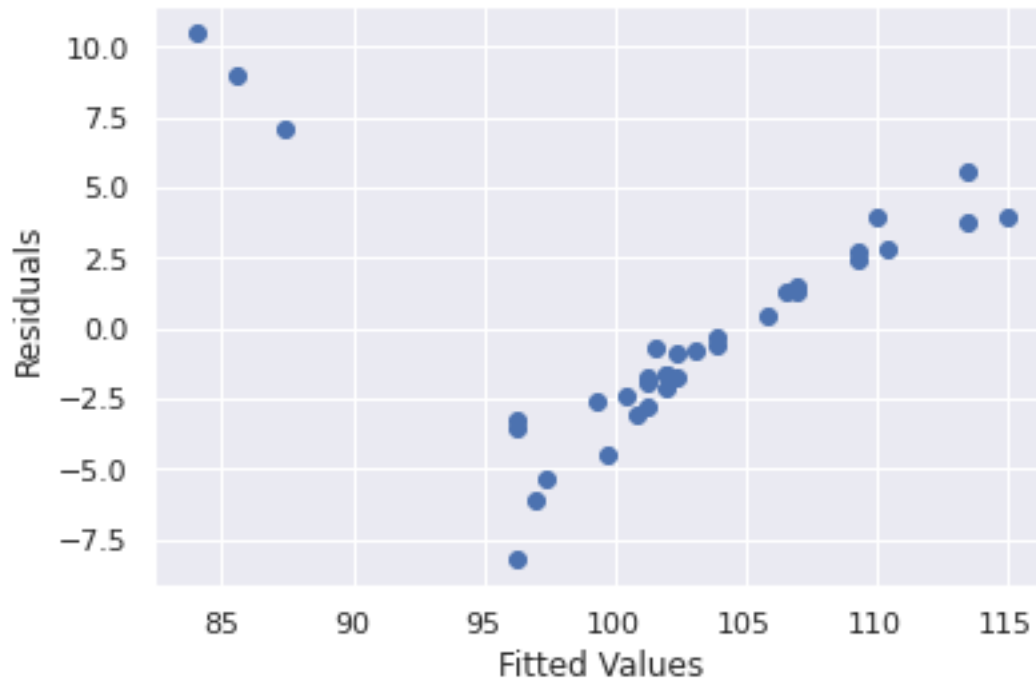
3.

We see clear pattern in the plot of residuals versus fitted values, suggesting the linear model is not adequate.

```

[5]: plt.scatter(olsBonds.fittedvalues, olsBonds.resid)
      plt.xlabel('Fitted Values')
      plt.ylabel('Residuals')
      plt.show()

```



4.

Recall that the 95% confidence interval is $\hat{\beta}_1 \pm z_{0.025} \text{SE}(\hat{\beta}_1)$, where the standard error is

$$\text{SE}(\hat{\beta}_1) \approx \frac{\hat{\sigma}}{s_x \sqrt{n-1}}$$

and

$$\hat{\sigma}^2 = \text{RSS}/(n-2).$$

Below is the numerical value. Here the normal distribution is used. If we use t -distribution instead, the confidence interval will exactly match the one reported in the summary above. As $n > 30$, the differences are tiny.

```
[6]: import numpy as np
from scipy.stats import norm, t

se = np.sqrt(olsBonds.ssr/(n-2))/(np.std(bonds['CouponRate'], ddof=1)*np.
    ↳sqrt(n-1))
# z = t.ppf(1 - 0.025, n-2)
z = norm.ppf(1 - 0.025)

(olsBonds.params['CouponRate'] - (z*se), olsBonds.params['CouponRate'] + (z*se))
```

```
[6]: (2.464778252846198, 3.6674260585957033)
```

The derivation of the confidence interval is based on the assumption that the linear model is adequate, which is not the case for the data set here. Thus the confidence interval is not meaningful.

Question Two

1.

Read in the data and drop the id and bond_id columns which clearly have no predictive power.

```
[7]: data = pd.read_csv('bonddata.txt', sep=',')
data.head()
```

```
[7]:      id  bond_id  trade_price  weight  current_coupon  time_to_maturity  \
0    10         1    126.268   7.616851         5.950        25.940349
1    38         2    101.009   0.018373         3.550         0.326392
2   336         3    108.144   3.537380         4.200         2.340601
3   781         4    116.953   0.402523         4.875         7.367653
4  1172        14    133.210   0.210685        10.750         4.159301
```

```
      is_callable  reporting_delay  trade_size  trade_type  ...  \
0              0          12.540    150000.0         3  ...
1              0           9.327    450000.0         3  ...
2              0           0.102    160000.0         4  ...
3              0          52.093     10000.0         4  ...
4              0          13.722     25000.0         4  ...
```

```
      received_time_diff_last9  trade_price_last9  trade_size_last9  \
0              2862729          128.596        120000.0
1              532644          101.086        500000.0
2              96293          107.917        400000.0
3              92559          115.924         10000.0
4             6400208          127.500         25000.0
```

```
      trade_type_last9  curve_based_price_last9  received_time_diff_last10  \
0              2          127.561201        3621492
1              2          101.103180        538510
2              2          107.874965         96293
3              2          116.160420         97177
4              2          131.066998        6400208
```

```
      trade_price_last10  trade_size_last10  trade_type_last10  \
0          129.089        200000.0         2
1          101.070         25000.0         2
2          107.917        400000.0         4
3          116.458          2000.0         3
4          127.500         25000.0         4
```

```

    curve_based_price_last10
0          129.435587
1          101.102243
2          107.874965
3          116.342802
4          131.066998

```

[5 rows x 61 columns]

```
[8]: data.drop(['bond_id', 'id'], axis='columns', inplace=True)
```

From Kaggle’s description of the data, `is_callable` and any column with “type” in the column name is categorical. These columns need to be converted into dummy variables for regression. Calling `astype('category')` on those columns is not necessary. Once the conversion to dummy variables is done, `astype('category')` will not make any difference.

After the conversion, there are 82 predictors in the data.

```
[9]: import pandas as pd

catCols = ['is_callable'] + [colName for colName in data.columns if 'type' in
    ↪ colName]

# for colName in catCols:
#     data[colName] = data[colName].astype('category')

data = pd.get_dummies(data=data, columns=catCols)
data.shape
```

[9]: (1620, 82)

2.

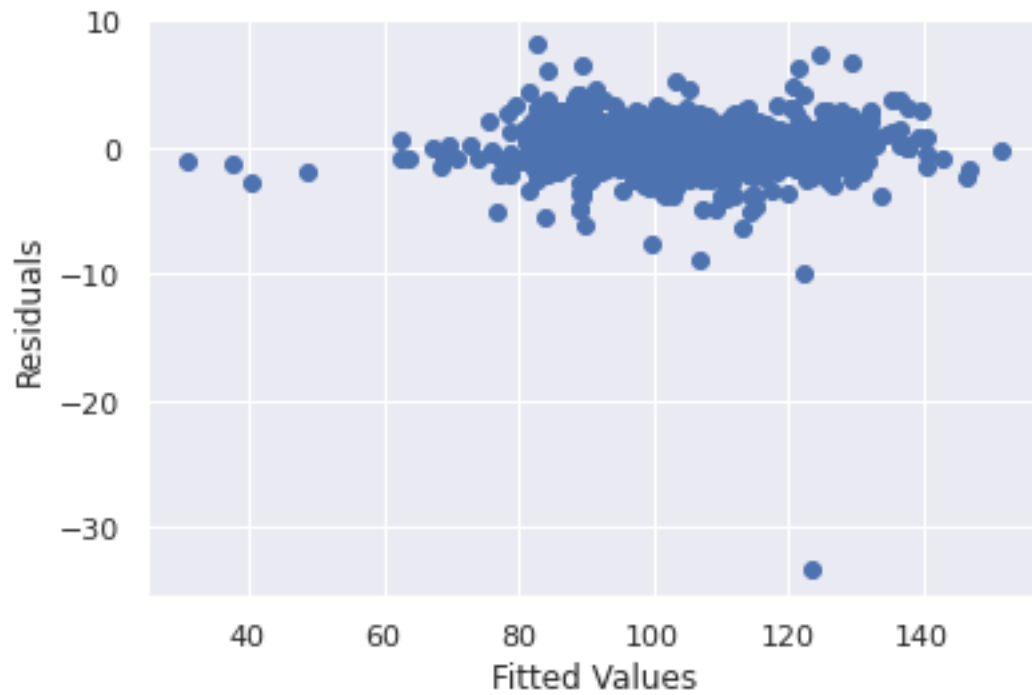
Below is the model fitted using all predictors. From the plot of residuals versus fitted values there seem to be a few outliers in the data.

```
[10]: import statsmodels.api as sm

X = sm.add_constant(data.drop('trade_price', axis='columns'))
fullmod = sm.OLS(endog=data['trade_price'], exog=X).fit()

plt.scatter(fullmod.fittedvalues, fullmod.resid)
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
```

```
plt.show()
```

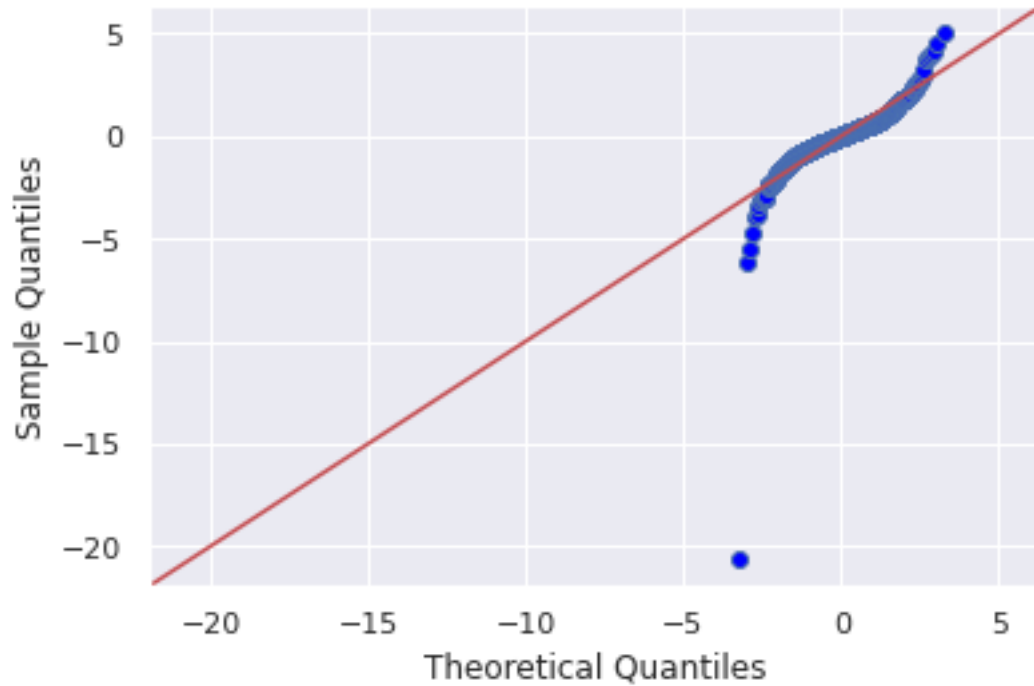


3.

From the below normal probability plot, it is clear the residual distribution has heavier tail than normal, and the left tail seems to be heavier. Also, there is a clear outlier in the data.

```
[11]: from statsmodels.graphics.gofplots import qqplot

qqplot(fullmod.resid, dist=norm, line='45', fit=True)
plt.show()
```



4., 5.

Here is the stepwise variable selection procedure using AIC/PRESS.

```
[12]: X.drop(['const'], axis='columns', inplace=True)
```

```
[13]: from statsmodels.stats import outliers_influence

def AIC(model):
    return model.aic

def PRESS(model):
    levs = outliers_influence.OLSInfluence(model).hat_matrix_diag
    return ((model.resid/(1 - levs))**2).sum()

def modelSelection(endog, exog, selectionMethod=AIC):
    """
    Assuming no constant column in the input exog
    """
    chosen = []
    remaining = list(exog.columns.values)
    best = np.inf
```

```

    for i in range(len(X.columns)):
        idx_quantities = [(idx, selectionMethod(sm.OLS(endog, exog=sm.
→add_constant(exog[[feature] + chosen])).fit())) for idx, feature in
→enumerate(remaining)]
        idx, q = min(idx_quantities, key=lambda idx_q: idx_q[1])
        if q >= best:
            break
        best = q
        chosen.append(remaining[idx])
        remaining = remaining[:idx] + remaining[idx+1:]

    return chosen

chosenAIC = modelSelection(endog=data['trade_price'], exog=X,
→selectionMethod=AIC)
chosenPRESS = modelSelection(endog=data['trade_price'], exog=X,
→selectionMethod=PRESS)

chosenAIC, chosenPRESS

```

```

[13]: (['trade_price_last1',
        'curve_based_price',
        'trade_type_2',
        'trade_price_last7',
        'trade_type_3',
        'trade_price_last10',
        'trade_price_last4',
        'curve_based_price_last5',
        'trade_type_last1_3',
        'trade_type_last4_3',
        'trade_type_last4_2',
        'trade_type_last1_2',
        'trade_price_last9',
        'trade_type_last9_2',
        'curve_based_price_last1',
        'curve_based_price_last4',
        'curve_based_price_last3',
        'received_time_diff_last10',
        'trade_type_last5_3',
        'trade_price_last3',
        'is_callable_1',
        'reporting_delay'],
        ['trade_price_last1',
        'curve_based_price',
        'trade_type_2',
        'trade_price_last7',

```

```

'trade_type_4',
'trade_price_last10',
'trade_type_last1_3',
'curve_based_price_last5',
'trade_price_last4',
'trade_type_last4_3',
'trade_type_last1_2',
'trade_type_last4_2',
'trade_size_last7',
'is_callable_1',
'trade_size',
'received_time_diff_last10',
'trade_price_last9',
'trade_type_last9_2',
'reporting_delay',
'trade_type_last5_3',
'received_time_diff_last8',
'trade_size_last4',
'trade_size_last5',
'trade_type_last1_4',
'trade_type_3'])

```

The selection process is computationally intense. We copy and paste the results below. Using AIC, the procedure selected 22 features, while using PRESS, it selected 25. There are 18 common features selected by both methods.

```

[14]: chosenAIC = ['trade_price_last1', 'curve_based_price', 'trade_type_2',
↳ 'trade_price_last7', 'trade_type_3', 'trade_price_last10',
↳ 'trade_price_last4', 'curve_based_price_last5', 'trade_type_last1_3',
↳ 'trade_type_last4_3', 'trade_type_last4_2', 'trade_type_last1_2',
↳ 'trade_price_last9', 'trade_type_last9_2', 'curve_based_price_last1',
↳ 'curve_based_price_last4', 'curve_based_price_last3',
↳ 'received_time_diff_last10', 'trade_type_last5_3', 'trade_price_last3',
↳ 'is_callable_1', 'reporting_delay']
chosenPRESS = ['trade_price_last1', 'curve_based_price', 'trade_type_2',
↳ 'trade_price_last7', 'trade_type_4', 'trade_price_last10',
↳ 'trade_type_last1_3', 'curve_based_price_last5', 'trade_price_last4',
↳ 'trade_type_last4_3', 'trade_type_last1_2', 'trade_type_last4_2',
↳ 'trade_size_last7', 'is_callable_1', 'trade_size',
↳ 'received_time_diff_last10', 'trade_price_last9', 'trade_type_last9_2',
↳ 'reporting_delay', 'trade_type_last5_3', 'received_time_diff_last8',
↳ 'trade_size_last4', 'trade_size_last5', 'trade_type_last1_4', 'trade_type_3']

len(chosenAIC), len(chosenPRESS), len(set(chosenAIC) & set(chosenPRESS))

```

```

[14]: (22, 25, 18)

```

Below are the summary of both models. They both have some features with coefficients very close

to zero. If we take away those features, the resulting models will be almost identical.

```
[15]: model = sm.OLS(endog=data['trade_price'], exog=sm.add_constant(X[chosenAIC])).
      ↪fit()
      model.summary()
```

```
[15]: <class 'statsmodels.iolib.summary.Summary'>
      """
```

```

                                OLS Regression Results
=====
Dep. Variable:                trade_price    R-squared:                0.980
Model:                        OLS           Adj. R-squared:        0.980
Method:                      Least Squares  F-statistic:             3527.
Date:                        Thu, 15 Oct 2020 Prob (F-statistic):       0.00
Time:                        09:06:58       Log-Likelihood:         -3092.3
No. Observations:            1620          AIC:                  6231.
Df Residuals:                1597          BIC:                  6355.
Df Model:                    22
Covariance Type:             nonrobust
=====
=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
const                1.2270    0.452      2.713    0.007
0.340    2.114
trade_price_last1    0.4340    0.028    15.456    0.000
0.379    0.489
curve_based_price    0.3929    0.038    10.296    0.000
0.318    0.468
trade_type_2        -0.8080    0.100    -8.112    0.000
-1.003    -0.613
trade_price_last7    0.1129    0.023     4.843    0.000
0.067    0.159
trade_type_3         0.8038    0.100     8.027    0.000
0.607    1.000
trade_price_last10   0.0376    0.026     1.422    0.155
-0.014    0.090
trade_price_last4    0.1033    0.030     3.392    0.001
0.044    0.163
curve_based_price_last5 -0.2031    0.050    -4.037    0.000
-0.302    -0.104
trade_type_last1_3   -0.2174    0.100    -2.169    0.030
-0.414    -0.021
trade_type_last4_3   -0.2073    0.103    -2.017    0.044
-0.409    -0.006

```

trade_type_last4_2	0.2007	0.104	1.929	0.054
-0.003	0.405			
trade_type_last1_2	0.2293	0.106	2.157	0.031
0.021	0.438			
trade_price_last9	0.0763	0.029	2.641	0.008
0.020	0.133			
trade_type_last9_2	0.1874	0.100	1.883	0.060
-0.008	0.383			
curve_based_price_last1	-0.0607	0.051	-1.180	0.238
-0.162	0.040			
curve_based_price_last4	0.2077	0.074	2.795	0.005
0.062	0.354			
curve_based_price_last3	-0.1591	0.066	-2.419	0.016
-0.288	-0.030			
received_time_diff_last10	-3.692e-08	2.06e-08	-1.791	0.074
-7.74e-08	3.52e-09			
trade_type_last5_3	-0.1440	0.088	-1.628	0.104
-0.317	0.029			
trade_price_last3	0.0469	0.031	1.534	0.125
-0.013	0.107			
is_callable_1	0.1791	0.120	1.498	0.134
-0.055	0.413			
reporting_delay	-1.75e-06	1.22e-06	-1.429	0.153
-4.15e-06	6.53e-07			

Omnibus:	1930.311	Durbin-Watson:	1.978
Prob(Omnibus):	0.000	Jarque-Bera (JB):	907009.858
Skew:	-5.538	Prob(JB):	0.00
Kurtosis:	118.389	Cond. No.	3.10e+07

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.1e+07. This might indicate that there are strong multicollinearity or other numerical problems.

"""

```
[16]: model = sm.OLS(endog=data['trade_price'], exog=sm.add_constant(X[chosenPRESS])).
      ↪ fit()
      model.summary()
```

```
[16]: <class 'statsmodels.iolib.summary.Summary'>
      """
```

```

                                OLS Regression Results
=====
Dep. Variable:                  trade_price    R-squared:                  0.980
```

```

Model:                      OLS      Adj. R-squared:      0.979
Method:                    Least Squares      F-statistic:      3361.
Date:                      Thu, 15 Oct 2020      Prob (F-statistic):      0.00
Time:                      09:06:58      Log-Likelihood:      -3094.8
No. Observations:          1620      AIC:      6238.
Df Residuals:              1596      BIC:      6367.
Df Model:                  23
Covariance Type:           nonrobust

```

```

=====
=====

```

	coef	std err	t	P> t
[0.025 0.975]				

const	0.7032	0.274	2.564	0.010
0.165 1.241				
trade_price_last1	0.4291	0.026	16.782	0.000
0.379 0.479				
curve_based_price	0.3540	0.028	12.554	0.000
0.299 0.409				
trade_type_2	-0.5763	0.114	-5.042	0.000
-0.800 -0.352				
trade_price_last7	0.1113	0.023	4.748	0.000
0.065 0.157				
trade_type_4	0.2491	0.105	2.383	0.017
0.044 0.454				
trade_price_last10	0.0486	0.026	1.853	0.064
-0.003 0.100				
trade_type_last1_3	0.0175	0.107	0.163	0.871
-0.193 0.228				
curve_based_price_last5	-0.1643	0.029	-5.654	0.000
-0.221 -0.107				
trade_price_last4	0.1400	0.025	5.677	0.000
0.092 0.188				
trade_type_last4_3	-0.2342	0.103	-2.272	0.023
-0.436 -0.032				
trade_type_last1_2	0.4564	0.120	3.810	0.000
0.221 0.691				
trade_type_last4_2	0.2174	0.106	2.060	0.040
0.010 0.424				
trade_size_last7	-8.056e-08	5.48e-08	-1.469	0.142
-1.88e-07 2.7e-08				
is_callable_1	0.1951	0.120	1.631	0.103
-0.040 0.430				
trade_size	7.488e-08	5.71e-08	1.312	0.190
-3.71e-08 1.87e-07				
received_time_diff_last10	-1.072e-07	6.32e-08	-1.697	0.090

-2.31e-07	1.67e-08				
trade_price_last9		0.0709	0.029	2.463	0.014
0.014	0.127				
trade_type_last9_2		0.1980	0.100	1.980	0.048
0.002	0.394				
reporting_delay		-1.664e-06	1.23e-06	-1.355	0.176
-4.07e-06	7.44e-07				
trade_type_last5_3		-0.1471	0.089	-1.655	0.098
-0.321	0.027				
received_time_diff_last8		8.794e-08	7.06e-08	1.245	0.213
-5.06e-08	2.27e-07				
trade_size_last4		-7.847e-08	5.67e-08	-1.385	0.166
-1.9e-07	3.26e-08				
trade_size_last5		6.841e-08	6.04e-08	1.133	0.258
-5.01e-08	1.87e-07				
trade_type_last1_4		0.2293	0.103	2.220	0.027
0.027	0.432				
trade_type_3		1.0304	0.109	9.426	0.000
0.816	1.245				
=====					
Omnibus:		1943.345	Durbin-Watson:		1.976
Prob(Omnibus):		0.000	Jarque-Bera (JB):		923080.057
Skew:		-5.607	Prob(JB):		0.00
Kurtosis:		119.402	Cond. No.		1.03e+21
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 2.04e-26. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

"""